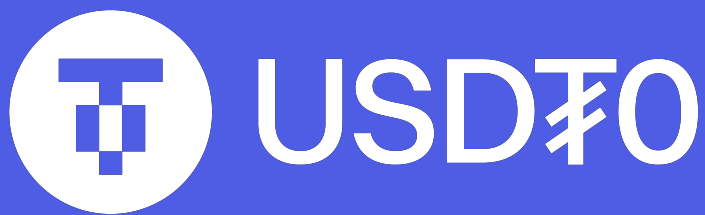


XAUt0 TON Multisig Transaction Re- Review



November 25, 2025

Table of Contents

Table of Contents	2
Scope	3
Overview	3
Failed Transactions Analysis	3
Execution Requirements	5
Critical Requirements	6
Conclusion	8
Appendix	9
Issue Classification	9

Scope

This report analyzes the original multisig orders (8–28) submitted to the multisig address EQACTgTTWCwWCaTSDJSY3inJL0SKJ_I_m4j2GNUPVyjHvaLU, which configure the XAUt0 OFT (Omnichain Fungible Token) contract on TON. It also examines the newly submitted multisig orders intended to re-execute operations that had previously failed.

Each transaction was decoded according to its corresponding LayerZero schema where applicable, validated for correctness, and mapped to the appropriate retry order.

Overview

An analysis of failed multisig orders #8-28 revealed two primary failure modes:

- Insufficient gas in deployment operations causing cascading failures
- Event emission errors that did not affect state updates.

Out of 21 failed transactions, only 11 require re-execution. Seven transactions successfully updated contract state despite execution errors, and their corresponding re-execution attempts should be skipped.

Failed Transactions Analysis

The following table presents an itemized analysis of all the failed transactions.

Order #	Operation	Chain	Failure Reason	Re-execution Order	Action Required
8	OP_SetPeer	Solana	Event emission error	-	None (state set successfully)
9	OP_DeployChannel	Solana	Insufficient gas	29 (completed)	None

Order #	Operation	Chain	Failure Reason	Re-execution Order	Action Required
10	OP_DeployConnection	Solana	Insufficient gas	30	Re-execute with increased gas
11	OP_SetPeer	Celo	Event emission error	-	None (state set successfully)
12	OP_DeployChannel	Celo	Insufficient gas	31	Re-execute with increased gas
13	OP_DeployConnection	Celo	Insufficient gas	32	Re-execute with increased gas
14	OP_SetPeer	Ink	Event emission error	-	None (state set successfully)
15	OP_DeployChannel	Ink	Insufficient gas	34	Re-execute with increased gas
16	OP_DeployConnection	Ink	Insufficient gas	33	Re-execute with increased gas
17	OP_SetEnforcedOptions	Solana	Event emission error	36	SKIP (already applied)
19	OP_SetEnforcedOptions	Celo	Event emission error	37	SKIP (already applied)
21	OP_SetEnforcedOptions	Ink	Event emission error	38	SKIP (already applied)
23	OP_SetLzConfig	Solana	Cascading failure	39	Re-execute after Phase 1
24	OP_SetLzConfig	Solana	Cascading failure	40	Re-execute after Phase 1

Order #	Operation	Chain	Failure Reason	Re-execution Order	Action Required
25	OP_SetLzConfig	Celo	Cascading failure	42	Re-execute after Phase 1
26	OP_SetLzConfig	Celo	Cascading failure	43	Re-execute after Phase 1
27	OP_SetLzConfig	Ink	Cascading failure	44	Re-execute after Phase 1
28	OP_SetLzConfig	Ink	Wrong execution order	35	Re-execute after Phase 1

Execution Requirements

This section outlines the conditions required to execute the failed transactions that require re-execution.

Phase 1: Deployment Operations (Execute First)

The following transactions must be executed before the rest. Though they can be executed in any order within Phase 1.

Order #	Operation	Chain
30	DeployConnection	Solana
31	DeployChannel	Celo
32	DeployConnection	Celo
33	DeployConnection	Ink
34	DeployChannel	Ink

Phase 2: Configuration Operations (Execute After Phase 1)

The following transactions must be executed after phase 1.

Order #	Operation	Chain	Dependency
35	SetLzConfig Receive	Ink	Phase 1 complete
39	SetLzConfig Send	Solana	Phase 1 complete
40	SetLzConfig Receive	Solana	Phase 1 complete
42	SetLzConfig Send	Celo	Phase 1 complete
43	SetLzConfig Receive	Celo	Phase 1 complete
44	SetLzConfig Send	Ink	Phase 1 complete

Orders to Skip (Do Not Execute)

The following transactions must not be executed.

Order #	Operation	Chain	Reason
36	SetEnforcedOptions	Solana	Already applied in state
37	SetEnforcedOptions	Celo	Already applied in state
38	SetEnforcedOptions	Ink	Already applied in state
41	SetLzConfig Receive	Solana	Duplicate of Order #40

Critical Requirements

Strict phase ordering is mandatory. Phase 2 operations depend on completed Phase 1 deployments. The original failures occurred because configuration operations attempted to execute on contracts which were not deployed with `deployChannel` / `deployConnection`, causing "no_state" errors.

All deployment operations must be provided sufficient gas to complete execution. Before executing Phase 2, it should be verified that all Phase 1 operations have completed successfully.

Conclusion

The re-execution strategy corrects insufficient gas allocation and maintains proper dependency ordering. Orders that appeared to fail to due to event emission correctly applied state updates and do not require re-execution. Skipping the three `SetEnforcedOptions` orders avoids redundant state updates.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.